Tree-Based Machine Learning Methods

Prediction, Inference, and Variable Selection

Slide Guide

Hemant Ishwaran and Min Lu | University of Miami

Guide structure

Part	Module	Slides
I	Training	40
II	Inference and Prediction	31
III	Variable Selection	38
IV	Advanced Topics	41

Part I

Training

Slides 1-40

Section map

- Opening and workshop ecosystem (Slides 1-6)
- Iowa housing quick start (Slides 7-21)
- Regression and quantile regression (Slides 22-28)
- Classification with the glioma data (Slides 29-33)
- Survival forests with the PBC data (Slides 34-38)
- Closing (Slides 39-40)

Opening and workshop ecosystem (Slides 1-6)

Slides 1-2: Course title and software ecosystem

Part I opens the workshop and immediately locates randomForestSRC within a broader ecosystem. RF-SRC supplies unified forests for regression, classification, survival, and competing risks; VarPro supplies observed-data variable priority; SGT supplies multivariate geometric tree splits; and RHF extends forest modeling to time-varying hazard estimation. The point of the opening is that the workshop is centered on one coherent family of R tools rather than a collection of unrelated examples.

Slide 3: Package and software links

This slide provides the concrete reference links for the workshop ecosystem: randomForestSRC on CRAN, the randomForestSRC.run visualization companion on GitHub, varPro on CRAN and GitHub, randomForestSGT on CRAN and GitHub, and randomForestRHF on CRAN and GitHub. It serves as the installation and documentation map students can return to during and after the course.

Slide 4: Four-part workshop outline

The workshop is organized into training, inference and prediction, variable selection, and advanced examples. Part I establishes the basic grow workflow and examples in regression, classification, and survival. The roadmap also previews OOB inference, restore mode, partial plots, VIMP, VarPro, class imbalance, missing-data imputation and OOD scoring, Super Greedy Trees, and Random Hazard Forests.

Slide 5: What a random forest is

A random forest is defined as an average of randomized tree predictors. The random vectors encode the resampling instructions and random feature selection used to grow each tree. This ensemble definition is the conceptual foundation for the package arguments introduced next and for the inbag-versus-OOB distinction developed in Part II.

Slide 6: Unified formula interface across families

The central interface is rfsrc(formula, data, ntree, mtry, nodesize). The table shows that the response specification determines the statistical family: ordinary outcomes for regression or classification, Surv objects for survival and competing risks, Multivar or cbind responses for multivariate analysis, and no response for unsupervised forests. Specialized front ends such as quantreg, imbalanced, and sidClustering use the same general design.

Iowa housing quick start (Slides 7-21)

Slide 7: Ames Iowa housing data

The quick start uses the Ames housing data as a realistic mixed-type regression problem. The target is SalePrice and the data contain 2,930 properties and 81 columns describing lot, structure, quality, age, garage, neighborhood, and sale characteristics. The example is large enough to show the practical advantages of a unified formula interface without requiring extensive preprocessing.

Slides 8-11: Grow a forest and inspect OOB performance

The flowchart introduces the three-stage workflow: grow a forest, check the output, and make predictions. These slides carry out the first two stages with rfsrc(SalePrice ~ ., data = housing). The printed object reports the forest settings and ends with OOB mean squared error and OOB R-squared. These are internally cross-validated quantities based on observations that were not used to grow the corresponding trees; the prediction step is completed on slide 20.

Slide 12: Transforming the outcome

SalePrice is refit after a logarithmic transformation. The slide emphasizes that a monotone outcome transformation need not materially alter the broad forest structure, but it does change the scale and interpretation of mean squared error. The before-and-after output makes this concrete: the raw-price MSE is very large because it is measured in squared dollars, whereas the log-scale error is numerically compact.

Slides 13-19: Reading the printed forest output

These slides map each major line of the printed object back to its controlling argument. They identify ntree = 500, nodesize = 5, the regression default mtry of one third of the variables, sampling without replacement (swor) with approximately 63.2% of observations per tree, the RF-R/regr family labels, MSE splitting, and nsplit = 10 random candidate cut points. The sequence teaches students to use print output as a compact audit of exactly what forest was grown.

Slide 20: Prediction on new data

The final quick-start step calls predict on the fitted forest and supplies the first ten housing records as newdata. The resulting object stores test-data predictions in $predicted. This establishes the basic training-versus-prediction distinction that Part II later develops in more detail.

Slide 21: General call to rfsrc

The interface diagram consolidates the main rfsrc arguments into one reference slide. Students should read it as the package-level map for formula specification, tree size, random feature selection, resampling, splitting rules, performance options, and objects returned by the fit.

Regression and quantile regression (Slides 22-28)

Slide 22: Nonparametric regression families

The family table is revisited with regression and quantile regression highlighted. This transition reinforces that ordinary regression forests and quantile forests share the same data and formula conventions, while differing in the terminal-node target and potentially in the splitting rule.

Slide 23: Two approaches to quantile regression forests

The slide contrasts Meinshausen-style quantile forests with the randomForestSRC implementation. The former grows trees with MSE splitting and constructs a terminal-node conditional distribution. randomForestSRC can instead use pinball-loss-oriented splitting and a local conditional distribution estimator. The distinction is between adapting only the post-tree distribution estimate and adapting both tree construction and local estimation to quantiles.

Slides 24-25: Iowa housing quantile regression

Three split rules are compared on the housing data: mse, quantile.regr, and the default la.quantile.regr. The quantile forest is printed and plot.quantreg displays the estimated conditional quantiles. The next slide enlarges the graphic so students can focus on how the estimated distribution changes across cases rather than only on a conditional mean.

Slide 26: The randomForestSRC.run overview workflow

The companion run.rfsrc function is introduced as a higher-level workflow. It fits a user-specified forest, performs tuning and diagnostics, and organizes performance and variable-importance graphics for regression, classification, survival, competing risks, and multivariate outcomes. The slide also shows the GitHub installation route for this workshop-oriented package.

Slides 27-28: Integrated housing analysis with run.rfsrc

A single run.rfsrc call is applied to SalePrice. The resulting multi-panel display collects the major components of an applied forest analysis in one place, allowing students to move from a valid basic fit to a more complete diagnostic and visualization workflow without manually assembling every plot.

Classification with the glioma data (Slides 29-33)

Slide 29: Classification family transition

The family table now highlights classification and the imbalanced two-class extension. The formula response is a factor for ordinary multiclass classification, while imbalanced provides a specialized interface for two-class problems with unequal class frequencies.

Slide 30: Glioma molecular profiling data

The classification example uses a subset of the Ceccarelli et al. glioma data. It contains 880 tissue samples, 1,241 predictors, and seven molecular subtype labels. The table illustrates the mix of methylation probes, molecular indicators, tumor grade, age, and gender, making this a genuinely high-dimensional multiclass problem.

Slide 31: Default multiclass forest and OOB confusion matrix

Because the response is a factor, rfsrc automatically fits a classification forest with Gini splitting. The overall OOB misclassification rate is about 7%, but the confusion matrix shows why class-specific errors matter: common classes are predicted very well, while G-CIMP-low and LGm6-GBM have much larger errors. The slide teaches students not to reduce a multiclass result to one overall number.

Slides 32-33: Integrated glioma analysis

run.rfsrc is applied to the multiclass glioma outcome. The overview display brings the fit, performance summaries, and classification-oriented diagnostics together. The second slide enlarges the graphic for interpretation.

Survival forests with the PBC data (Slides 34-38)

Slide 34: Survival family transition

The family table shifts attention to right-censored survival. A Surv(time, status) response tells rfsrc to grow a random survival forest, with survival-specific terminal-node estimators and split rules.

Slide 35: Mayo Clinic primary biliary cirrhosis data

The PBC data provide the survival example. The study includes trial and follow-up patients with time, endpoint status, treatment, demographics, clinical findings, and laboratory measurements. The R data have 418 rows and 20 columns before the ID is removed and complete-case analysis is applied.

Slide 36: Canonical random survival forest

The original status variable is saved in pbc.cr, then positive endpoint codes are collapsed so transplant or death is treated as an event. The Surv(time, status) response identifies the survival family. The printed object reports 276 complete cases, 129 events, log-rank splitting, OOB CRPS, standardized CRPS, and the requested performance error.

Slides 37-38: Integrated PBC survival analysis

run.rfsrc is called with the PBC survival formula. The resulting overview summarizes the fitted survival forest and its major diagnostics in one display; the continuation slide enlarges that output.

Closing (Slides 39-40)

Slide 39: Workshop roadmap and transition

The outline closes Part I and positions the remaining modules. Part II moves from growing forests to OOB inference, performance assessment, prediction, restore mode, and partial plots; Part III addresses VIMP, uncertainty, minimal depth, and VarPro; Part IV develops class imbalance, imputation and OOD scoring, SGT, and RHF.

Slide 40: References

The module closes with the primary references for Breiman random forests, the randomForestSRC getting-started material, the Ames housing data, the glioma study, multiclass AUC splitting, CART, random survival forests, the PBC data, and survival split rules.

Part II

Inference and Prediction

Slides 1-31

Section map

- Inbag and out-of-bag inference (Slides 1-9)
- Prediction error (Slides 10-14)
- Prediction and restore mode (Slides 15-23)
- Partial plots (Slides 24-29)
- Closing (Slides 30-31)

Inbag and out-of-bag inference (Slides 1-9)

Slide 1: Part II title

Part II changes the emphasis from growing a forest to using the fitted object. The central distinction is between the full inbag ensemble used for prediction and the case-specific OOB ensemble used for inference and internally cross-validated evaluation.

Slide 2: Terminal-node statistics and predictions by family

The table summarizes what each family stores in a terminal node and how those statistics are aggregated into predictions. Examples include means for regression, class proportions and classifiers for classification, Kaplan-Meier survival and Nelson-Aalen cumulative hazard for survival, and cause-specific cumulative incidence and hazard quantities for competing risks. Multivariate families repeat the appropriate construction response by response.

Slide 3: A random forest produces two ensembles

The inbag ensemble averages every trained tree and is evaluated at new covariate values. The OOB ensemble is different for every training case because it averages only trees for which that case was excluded from the resample; approximately 36.8% of the trees contribute for a given case. This distinction is the foundation for honest training-sample inference.

Slide 4: OOB generalization error

Because each OOB prediction is formed without using that observation to grow the contributing trees, the OOB ensemble behaves like a leave-one-out bootstrap estimator. Applying the relevant loss to these case-specific predictions estimates generalization error and supplies a convenient basis for forest tuning. The practical message is that a separate cross-validation loop is often unnecessary.

Slide 5: Key classification components

For classification, $predicted and $predicted.oob contain inbag and OOB class-probability matrices, while $class and $class.oob contain the corresponding class labels. Students should use the OOB versions when evaluating the fitted model on the training observations.

Slide 6: Glioma example: inbag versus OOB error

The glioma forest illustrates the difference sharply. Inbag class predictions reproduce the training labels with zero misclassification, whereas the OOB error is about 7.1%. The confusion matrix and OOB metrics therefore describe realistic predictive behavior; the perfect inbag result mainly reflects the flexibility of the trained forest.

Slide 7: Inspecting probability and class outputs

The first five rows of the inbag and OOB probability matrices are printed together with their class predictions. The inbag probabilities are generally more concentrated because the cases helped train those trees. The OOB probabilities are the appropriate inputs for training-sample calibration, error, and case-level inference.

Slide 8: Key survival components

For survival, $time.interest defines the common event-time grid. The fitted object stores inbag and OOB mortality scores, survival matrices, and cumulative-hazard matrices. Every survival curve and time-dependent summary is indexed by the same time grid.

Slide 9: PBC inbag and OOB survival curves

Three PBC patients are selected and their inbag and OOB survival curves are overlaid. The plot makes the ensemble distinction visible at the case level: the two curves can differ because only the OOB curve excludes trees trained on that patient.

Prediction error (Slides 10-14)

Slide 10: Default error measures by family

The table maps statistical families to their default prediction-error measures. Regression uses mean squared error; classification uses misclassification and Brier-type measures; imbalanced classification emphasizes G-mean; and survival and competing risks use one minus Harrell concordance as the requested performance error. Multivariate objects return response-specific errors.

Slide 11: Classification error helpers

The chosen metric is stored through $err.rate, but the OOB responses and probability or class predictions permit other measures to be recomputed. The package helpers shown are get.misclass.error, get.brier.error, get.logloss, and get.auc. This encourages reporting a metric that matches the scientific use of the classifier rather than relying on a single default.

Slide 12: Multiple glioma performance measures

The glioma example reports OOB Brier score, normalized Brier score, AUC, log-loss, overall misclassification, and class-specific errors. tail(o$err.rate, 1) extracts the final tree-wise error row. The slide demonstrates that discrimination, probability accuracy, and hard classification answer different questions and need not rank models identically.

Slide 13: Survival error options

Three survival measures are contrasted. Harrell C-error is intuitive but computationally costly and can be affected by heavy censoring. The time-varying Brier score measures prediction error over follow-up and is presented as computationally inexpensive and robust. Time-varying AUC measures discrimination, with get.auct.survival identified as the calculation interface. The choice separates overall ranking, prediction accuracy, and discrimination at particular times.

Slide 14: PBC time-varying Brier score and AUC

The PBC figure displays prediction performance over follow-up time rather than reducing the model to one number. The Brier curve tracks time-specific prediction error, while the AUC-t curve tracks time-specific discrimination. The slide identifies plot.brier.auc in randomForestSRC.run as the function used for the display and also points to plotBrierAUC in randomForestSRC.

Prediction and restore mode (Slides 15-23)

Slide 15: Four uses of prediction

Prediction appears in four forms: scoring ordinary test data, scoring deliberately modified data for partial plots, substituting new outcomes into a trained forest through outcome = "test", and restoring a forest without supplying newdata. The slide separates these uses before the common predict interface is examined.

Slide 16: General call to predict

The interface diagram serves as the predict reference. It organizes newdata, outcome handling, performance calculations, imputation, importance, proximity, forest weights, tree extraction, and other restore-mode options around the fitted forest object.

Slide 17: Veterans lung-cancer trial

The canonical prediction example uses the Veterans Administration lung-cancer trial, with 137 patients and variables for treatment, cell type, survival time and status, Karnofsky score, diagnosis time, age, and prior therapy. The example is useful because it includes both survival outcomes and categorical predictors.

Slide 18: Creating unequal train and test samples with factors

The data are converted to factors where appropriate and split 25/75 into training and test sets. The summaries show that the two sets share the factor-level definitions even though the observed labels and frequencies differ. This is the ordinary setting in which factor-valued test data can be routed through a trained forest.

Slide 19: Training and scoring the veteran data

A survival forest is grown on 34 training cases and used to score 103 test cases. Separate printed summaries distinguish OOB training performance from performance computed using the test outcomes. The example reinforces that the test prediction object retains the grow-forest settings while reporting the size and performance of the supplied test data.

Slide 20: A previously unseen factor level

A deliberately new celltype level is inserted into test data. The package does not discard the record; instead, the unrecognized level is represented as missing and handled by the forest's missing-data mechanism. Printing $xvar verifies this behavior and makes the edge case explicit.

Slide 21: What restore mode can recover

Calling predict(object, ...) without newdata restores the original training forest and can request additional calculations after tuning. The slide lists VIMP, forest weights, proximities, distances, browser-based tree extraction, variable-use counts, and split-depth summaries. Restore mode avoids regrowing the forest merely to obtain a new diagnostic.

Slide 22: Restoring the glioma forest with another performance target

The trained glioma object is restored with perf.type = "brier". The tree structure is unchanged, but the requested performance output is recomputed on the OOB ensemble using the Brier-oriented target. This demonstrates how one fitted forest can support several post-training summaries.

Slide 23: Constructing predictions from forest weights

For mtcars regression, OOB forest weights are extracted and multiplied by the observed response. The reconstructed predictions agree with $predicted.oob up to numerical precision, and the resulting mean squared error matches the printed OOB error. The example shows that forest weights expose the ensemble as an explicit weighted estimator.

Partial plots (Slides 24-29)

Slide 24: Definition of a partial plot

A partial plot is presented as a prediction exercise. To study variable j at value x, every training case is copied, coordinate j is replaced by x, the modified rows are scored by the forest, and the predictions are averaged. Repeating this over x produces an adjusted marginal display of the fitted response surface.

Slide 25: General call to plot.variable

plot.variable is the high-level plotting interface for variable effects. The reference slide shows the main controls for variables, survival output type, time point, partial versus nonpartial displays, smoothing, and plot customization.

Slide 26: Peak VO2 survival data

The partial-plot example uses 2,231 systolic heart-failure patients with 39 predictors derived from baseline clinical measurements and exercise testing. The endpoint is all-cause death, and peak VO2 is a central exercise variable. The data frame contains 41 columns including the survival response fields.

Slide 27: Age partial effect on two-year survival

A random survival forest is fit and plot.variable is used to estimate the partial relationship between age and survival at two years. The plot averages over the observed values of all other predictors, so it is a forest-adjusted display rather than an unadjusted Kaplan-Meier comparison.

Slide 28: Smoothing the age partial plot

The same age partial effect is redrawn with smooth.lines = TRUE. The underlying forest predictions do not change; smoothing is used only to make the overall pattern easier to read.

Slide 29: Changing the survival output scale

The partial display is recomputed with surv.type = "rel.freq". This slide emphasizes that the same prediction machinery can be summarized on different survival-related scales, and that the analyst should label the vertical axis according to the quantity requested.

Closing (Slides 30-31)

Slide 30: Workshop roadmap and transition

The outline marks the end of inference and prediction. Part III turns to variable selection, beginning with permutation VIMP and uncertainty intervals and then moving to minimal depth and VarPro.

Slide 31: References

The references cover the .632+ bootstrap interpretation of OOB estimation, the Veterans Administration survival example, and the peakVO2 heart-failure study used for the partial-plot illustrations.

Part III

Variable Selection

Slides 1-38

Section map

- Permutation VIMP and uncertainty (Slides 1-14)
- Minimal depth (Slides 15-19)
- VarPro: observed-data variable priority (Slides 20-32)
- Individual VarPro and package overview (Slides 33-36)
- Closing (Slides 37-38)

Permutation VIMP and uncertainty (Slides 1-14)

Slides 1-2: Part III title and variable-selection roadmap

Part III asks which variables carry predictive information and how stable that conclusion is. The module proceeds through permutation variable importance, subsampling-based confidence intervals, minimal depth, and VarPro. The ordering moves from prediction-loss-based importance to tree-structure importance and then to observed-data rule-release importance.

Slide 3: Permutation importance

For each tree, the original OOB loss is compared with the loss after the values of variable j are shuffled among that tree's OOB cases. Shuffling breaks the case-specific alignment of that feature with the other covariates and outcome. The increase in loss is the tree-level importance, and averaging across trees gives forest VIMP.

Slide 4: Formal OOB VIMP contrast

The equations write VIMP as perturbed OOB loss minus original OOB loss. A positive value means that breaking the variable's information harms prediction; a value near zero means little change; and negative values can occur because of sampling variation or because the perturbation happens to improve the selected loss in a finite forest.

Slide 5: How a permuted case moves through a fixed tree

The tree diagram keeps the tree structure fixed and changes only the tested coordinate. A case may therefore take a different branch at nodes that split on the permuted variable and arrive at a different terminal node. The resulting change in prediction contributes to the perturbed loss.

Slide 6: Three VIMP noising schemes

randomForestSRC provides anti-VIMP, Breiman-Cutler permutation VIMP, and random-daughter VIMP. importance = TRUE requests the anti method; importance = "permute" requests the literal permutation construction; and importance = "random" requests random noising. The remainder of the module emphasizes permutation VIMP.

Slide 7: Computing VIMP during grow, restore, or postprocessing

VIMP can be requested while the forest is grown, requested later through restore-mode predict, or calculated with vimp. block.size controls how trees are grouped for the calculation. The final example perturbs variables jointly, which is useful for correlated or scientifically grouped features and need not equal the sum of their individual VIMPs.

Slide 8: Iris individual and joint VIMP

The iris example shows overall and class-specific VIMP. Petal length and petal width dominate the individual rankings. When those two features are perturbed together, the joint importance is much larger than either individual value and is not additive, reflecting shared information and interaction in the classifier.

Slides 9-10: PeakVO2 VIMP illustration

The survival example includes 2,231 patients, 39 predictors, and 742 deaths over roughly five years of follow-up. The VIMP display identifies peak VO2, BUN, and exercise interval as the strongest signals. The repeated slide enlarges the plot so that the ranking and uncertainty around the leading and near-zero variables can be read clearly.

Slide 11: Subsampling confidence intervals for VIMP

Uncertainty is estimated by repeatedly sampling observations without replacement using m much smaller than n; the displayed rule of thumb gives m approximately equal to the square root of n, or about 48 here. The subsample variation supplies a standard error, and a normal approximation gives the confidence interval. The code fits the original forest, calls subsample, and draws the intervals with plot.subsample(oo, alpha = .05).

Slide 12: Regression VIMP with uncertainty: Iowa housing

The housing plot shows why intervals matter in a long predictor list. Overall quality is the dominant variable, followed by major size, garage, and age-of-home variables. Beyond the leading set, many estimates cluster near zero, so a raw complete ranking would suggest distinctions that are not supported by the uncertainty display.

Slide 13: Class-specific VIMP: glioma

The multiclass plot separates overall VIMP from subtype-specific importance. A probe can be modest in the global panel yet strongly informative for one glioma class, and unequal class sizes can produce different uncertainty across panels. The scientific target therefore determines whether the overall or class-specific result is most relevant.

Slide 14: General call to subsample

The subsample interface controls the number of replicates, the subsample ratio, outcome stratification, and an optional double-bootstrap route. Stratification is particularly important when rare classes or survival events must remain represented in each replicate.

Minimal depth (Slides 15-19)

Slide 15: Minimal depth as a structural importance measure

Minimal depth ranks a variable by how close to the root it first splits. A root or shallow split affects many observations, whereas a deep split acts only within a smaller subgroup. The method is fast, applies broadly across tree families, and does not depend on a prediction-loss function; its main difficulty is choosing an importance threshold.

Slide 16: Tree-depth illustration

The diagram labels the root as depth zero and shows several variables first entering at different depths. The statistic uses the first appearance of a variable in each tree, then averages over the forest. Consistently important variables tend to appear early and therefore have smaller minimal depth.

Slide 17: General call to max.subtree

max.subtree is the package interface for minimal-depth and maximal-subtree summaries. The reference slide also indicates how higher-order information can be requested when the analyst wants to investigate interactions rather than only first-order variable rankings.

Slide 18: PeakVO2 minimal-depth ranking

The first-order minimal depth is extracted, sorted, and drawn as a horizontal bar chart. Because the bars represent depth, shorter is better. Peak VO2 and exercise interval appear among the shallowest variables, broadly agreeing with the leading permutation-VIMP results despite the different definition of importance.

Slide 19: Guiding a second forest by variable-use counts

Variable-use counts from the original forest are extracted with restore-mode predict and supplied as xvar.wt in a second survival forest. The new minimal-depth ranking concentrates random feature selection on predictors that were used frequently before. The example illustrates a practical guided-forest strategy for focusing computation in a larger predictor set.

VarPro: observed-data variable priority (Slides 20-32)

Slide 20: Why permutation can be problematic under correlation

VarPro is motivated by a specific limitation of permutation VIMP. Shuffling one coordinate preserves that coordinate's marginal values but can destroy its dependence with the remaining features, creating covariate combinations with little or no support in the observed data.

Slide 21: A concrete artificial combination in peakVO2

One observed patient has peak VO2 equal to 4.2, BUN equal to 20, and exercise interval equal to 360. Permutation can replace only peak VO2 by 43.8, the sample maximum, while all other measurements remain fixed. The value itself is real, but the resulting patient profile may be implausible; prediction loss can then include an extrapolation penalty as well as the variable's true contribution.

Slide 22: Correlated-data geometry under permutation

The diagrams show a tree-rule region superimposed on correlated observations. Permuting a coordinate moves cases away from the observed joint pattern and can place them in poorly supported parts of the rule space. This visual explains why permutation VIMP can be unstable or misleading when correlation is strong.

Slide 23: Rule release uses only observed cases

VarPro changes the rule rather than changing the data. For a tree branch, the restriction on variable j is released while all other rule conditions remain fixed. The original and released regions are scored using observed cases, and the difference in a user-defined statistic measures the variable's local contribution. Aggregation over many rules produces global variable priority.

Slide 24: VarPro advantages and limitation

The stated advantages are speed, improved behavior in correlated problems, user-defined test statistics, no dependence on a prediction-error metric, user-specified or tree-guided rules, and sparsity. The trade-off is that the standardized priority value is mainly useful for ranking and selection rather than a simple increase-in-error interpretation.

Slide 25: General call to varpro

The interface slide organizes formula and data inputs together with rule-generation controls, sparsity, tree and node complexity, split-weight guidance, and parallel computation. It is the main reference for fitting the supervised global VarPro model.

Slide 26: Canonical VarPro fit on peakVO2

The basic survival call is followed by importance(o). In the displayed fit, BUN has the largest standardized score, followed by peak VO2 and exercise interval, with several additional clinical variables receiving smaller values. The table is intended first as a ranking and then as input to a selection rule.

Slide 27: Cross-validated VarPro cutoff

cv.varpro returns the ranked scores, a selected list, conservative and liberal alternatives, the error associated with candidate cutoffs, and the chosen z threshold. In the displayed run the minimum error occurs for an 11-variable model, while neighboring models have similar errors. The conservative and liberal lists make the parsimony-versus-screening trade-off explicit.

Slides 28-29: VIMP and VarPro side by side

Permutation VIMP with subsampling intervals is compared with cross-validated VarPro scores, first in the full displays and then with the VIMP axis enlarged. The numerical axes are not commensurate because the estimands differ. The useful comparison is the ranking: agreement among the leading variables is a robustness signal, while disagreements point to correlation, redundancy, local support, or uncertainty worth investigating.

Slide 30: High-dimensional van de Vijver breast-cancer data

The example has 78 patients and 4,707 columns, including survival time and censoring, so the expression dimension is far larger than the sample size. The historical study produced a 70-gene prognostic signature, which supplies a reference set for comparing guided VarPro analyses.

Slide 31: Guided rule generation in p much larger than n

Four strategies are compared: lasso guidance, lasso plus VIMP, lasso plus VIMP plus shallow-tree information, and lasso plus VIMP with sparsity turned off. The entire analysis is repeated 25 times, and each selected set is intersected with the original 70-gene signature. The split weights guide candidate rules; VarPro still scores variables through observed-data rule release.

Slide 32: Signature overlap and the recall-parsimony trade-off

The displayed intersections contain 9 signature genes for lasso alone, 12 for lasso plus VIMP, 13 after adding shallow-tree guidance, and 19 when sparsity is disabled. A longer overlap is not automatically better because it may come with a much larger selected set. The example demonstrates how guidance and sparsity alter recall in an extreme high-dimensional problem.

Individual VarPro and package overview (Slides 33-36)

Slide 33: Why individual importance is needed

A single global ranking can hide patient heterogeneity. iVarPro asks a local sensitivity question: around one observed case, how much does the target change when a given coordinate is varied within data-supported neighborhoods? The goal is a case-specific importance profile rather than one average ranking for the full cohort.

Slides 34-35: Individual importance and case-level partial display

The example grows a VarPro analysis for peakVO2, obtains case-specific importance with ivarpro, and calls plot on that object. The display relates individualized peak-VO2 importance to the observed peak-VO2 value, with color assigned by exercise interval and point size by the outcome. Slide 35 enlarges the same figure. The two slides show how local importance varies across the covariate range and across patient context, information that is absent from a single global ranking.

Slide 36: Overview of the varPro package

The package diagram connects tree-grown terminal-node rules to a common four-step rule-and-release ensemble: grow, extract, compute a statistic, and aggregate. The same machinery supports supervised global VarPro, unsupervised UVarPro, and individual iVarPro analyses.

Closing (Slides 37-38)

Slide 37: Workshop roadmap and transition

The outline marks completion of variable selection. Part IV moves to class imbalance, missing-data imputation and OOD scoring, Super Greedy Trees, and Random Hazard Forests.

Slide 38: References

The references cover permutation VIMP and subsampling inference, minimal depth, the rule-based Variable Priority method, the van de Vijver gene-expression study, individual VarPro, and unsupervised variable priority.

Part IV

Advanced Topics

Slides 1-41

Section map

- Class-imbalanced classification (Slides 1-13)
- Missing-data imputation and OOD scoring (Slides 14-23)
- Super Greedy Trees and forests (Slides 24-33)
- Random Hazard Forests (Slides 34-39)
- Closing (Slides 40-41)

Class-imbalanced classification (Slides 1-13)

Slides 1-2: Part IV title and advanced-example roadmap

Part IV develops four advanced extensions of the core forest workflow: two-class imbalance, train/test missing-data imputation with OOD scoring, Super Greedy Trees with richer split geometry, and Random Hazard Forests for longitudinal covariates. The module shows how the broader ecosystem builds on the training, OOB inference, prediction, and variable-selection ideas developed in Parts I-III.

Slides 3-4: Classification family and what imbalance means

The family table highlights the imbalanced two-class interface. The next slide defines the setting: one majority outcome is observed far more often than the minority outcome. Examples include customer churn, spam or software-error detection, industrial fault detection, and biomedical targets with low prevalence or short event windows.

Slides 5-6: Why overall error can be misleading

The esophageal-treatment overlap example has 6,649 surgery patients and 988 adjuvant-treatment patients, an imbalance ratio of about 6.8. The overall OOB error is only 10.4%, yet 694 of 988 adjuvant cases are misclassified. Majority specificity is 98.5%, minority sensitivity is 29.8%, and their geometric mean is only 54.2%, showing how the overall rate is dominated by the majority class.

Slide 7: Metrics for imbalanced performance

For imbalance-focused evaluation, the slide recommends G-mean and precision-recall AUC rather than relying on ordinary misclassification error or ROC AUC. The accompanying curves illustrate that a classifier can look acceptable on a majority-dominated metric while offering poor recovery of the target minority class.

Slides 8-9: From the 0.5 Bayes rule to the RFQ threshold

The usual equal-cost decision rule predicts class 1 only when its estimated probability exceeds 0.5. When the event prevalence is small, that rule favors the majority class. RFQ replaces 0.5 by the empirical event prevalence pi. The slide states that this prevalence threshold is both cost-weighted Bayes optimal and maximizes the sum of majority and minority correct-classification rates.

Slide 10: General call to imbalanced

The interface diagram summarizes the imbalanced formula, data, method, performance, and forest-control arguments. The glioma example that follows compares the ordinary random-forest classifier with the RFQ approach, using class-specific errors and imbalance-sensitive performance measures.

Slide 11: Constructing a binary imbalanced glioma outcome

Four common glioma subtypes are combined into a super-majority class and the remaining subtypes form the minority class. The new binary outcome contains 786 class-0 and 94 class-1 cases, producing an imbalance ratio of about 8.36.

Slide 12: Standard random-forest classifier

The standard Gini forest classifies all 786 majority cases correctly but misses 42 of 94 minority cases. Its overall error is under 5%, yet minority class error is 44.7% and G-mean is about 0.751. This is the same masking phenomenon seen in the treatment example.

Slide 13: RFQ classifier

The RFQ fit uses AUC splitting and the prevalence-based decision rule. In the displayed run, all 94 minority cases are recovered while 95 majority cases are labeled positive. The overall error increases, but G-mean rises to about 0.938 and PR-AUC also improves, illustrating why the chosen objective must reflect the minority-class goal.

Missing-data imputation and OOD scoring (Slides 14-23)

Slide 14: Where imputation can enter the workflow

Missing values can be handled during training or prediction with na.action = "na.impute", before analysis with impute, or through learned train/test imputation with impute.learn. The slide distinguishes on-the-fly imputation, missForest-style response-by-response forests, and banked training forests that later impute test data.

Slide 15: General call to impute

The interface graphic summarizes the principal impute arguments and provides a reference for supervised or unsupervised imputation, the missForest and grouped mForest variants, iteration controls, and the object returned for subsequent analysis.

Slide 16: On-the-fly imputation

OTFI computes split statistics from nonmissing values and assigns missing cases to daughter nodes by sampling from nonmissing inbag data. It can be supervised by supplying a response formula or unsupervised by using multivariate pseudo-responses. The examples apply both routes to PBC data.

Slide 17: missForest and grouped mForest

The missing-data problem is reframed as prediction: each variable with missing values becomes a response in a forest. mf.q = 1 requests the response-by-response missForest-style route. In wide data, mf.q can group variables into multivariate regressions, reducing the number of separate forests per cycle.

Slides 18-19: Learning an imputer for deployment

impute.learn creates a reusable fitted imputer; predict applies it to new data; save.impute.learn and load.impute.learn preserve the banked models. The full-slide schematic on slide 19 makes the train/test separation explicit: the imputation rule is learned once from training data and then carried forward rather than rebuilt from the deployment sample.

Slide 20: Airquality train/test imputation example

The airquality variables are split into 100 training rows and a test set. impute.learn is fit with iterative full sweeps and then applied to the test data. target.mode = "all" is recommended because it learns an imputation model for every variable, including variables that happened to be complete in this particular training split but may be missing later.

Slides 21-22: Three meanings of OOD and the scoring workflow

The OOD section separates three questions: whether a row is unusual overall, unusual on coordinates useful for prediction, or unusual for the predictive task itself. The proposed unifying question is whether each observed coordinate can be reconstructed from the others as well as it could be in training. Slide 22 turns this into a deployment schematic: learn the reconstruction relationships and reference distribution in training, then compare test-time reconstruction discrepancies with that reference.

Slide 23: OOD scores and training-reference percentiles

A supervised impute.learn fit is trained for Solar.R with save.ood = TRUE, and impute.ood scores the held-out rows. $score gives the raw row-level discrepancy and $score.percentile locates that discrepancy within the training reference distribution. The percentile is the more immediately interpretable quantity for flagging unusually difficult-to-reconstruct rows.

Super Greedy Trees and forests (Slides 24-33)

Slide 24: Why geometric splits are useful

CART splits one variable at a time, so a multivariate boundary may require many rectangular cuts and a deep, unstable tree. Super Greedy Trees instead use rich geometric cuts that can encode combinations of variables directly. CART remains a special case, but SGT can represent local linear, quadratic, and interaction-based structure in a single node split.

Slide 25: Lasso geometric split

At a node, SGT builds a dictionary of selected coordinates, nonlinear transforms, and interactions, then solves a lasso-penalized loss problem. Candidate daughters are formed by thresholding the fitted score. With main effects and interactions, the resulting forest prediction can be written as a local polynomial whose coefficients vary with the case.

Slide 26: Examples of SGT split geometry

The visual compares the boundary families available to SGT, progressing from axis-aligned CART cuts to hyperplanes, curved quadratic forms, and higher-order interaction surfaces. The purpose is to show that one node can encode a relationship that would require many ordinary CART splits.

Slide 27: The hcut hierarchy

randomForestSGT indexes split dictionaries with hcut. hcut = 0 gives CART, hcut = 1 gives a linear hyperplane, hcut = 2 adds quadratic terms, hcut = 3 adds pairwise interactions, and larger values add higher-degree polynomials or higher-order interactions. treesize and nodesize control complexity, while filter, tune.hcut, and pure.lasso control dimension reduction and the use of lasso-based splitting.

Slide 28: Fitted surfaces as hcut increases

The contour display shows how the estimated surface becomes more flexible as the split dictionary expands. The visual is meant to connect the abstract hcut index to the geometry the forest can represent.

Slide 29: Tuning hcut in Friedman 1 data

Friedman 1 data are simulated with 50 additional noise variables. tune.hcut fits shallow forests over candidate hcut values up to 3 and returns a filter object containing the selected split family and basis functions. That object can be passed directly to rfsgt so tuning and final fitting use the same screened dictionary.

Slide 30: Using the tuned split family

The tuned forest selects hcut = 3 with 14 basis dimensions and only five retained variables. Its OOB R-squared is about 0.981. The result illustrates how a compact interaction-rich split can recover the nonlinear Friedman surface despite many noise variables.

Slide 31: CART and hyperplane comparisons

The same tuned filter is forced to hcut = 0 and hcut = 1. CART gives OOB R-squared about 0.918 and the hyperplane forest about 0.923, both well below the hcut = 3 result. The comparison isolates the benefit of allowing interaction-rich geometry rather than attributing the gain to a different variable screen.

Slides 32-33: SGT as a local model explainer

A shallow pure-lasso SGT forest is fit and get.beta extracts OOB predictions, local coefficients, and per-term partial contributions. The displayed predictions match the forest's OOB predictions exactly. The coefficient matrix describes the local polynomial used for each case, while the partial matrix multiplies those coefficients by the corresponding basis values to show how each term contributes to the prediction.

Random Hazard Forests (Slides 34-39)

Slide 34: Dynamic hazard modeling

Baseline survival models use one covariate snapshot even when physiology, medications, laboratory values, devices, exposures, and symptoms change during follow-up. RHF models the hazard along a longitudinal covariate trajectory. The canonical formula is Surv(id, start, stop, event) ~ ., making subject identity and interval structure explicit.

Slide 35: Counting-process data format

Each subject can contribute repeated rows. id identifies the subject, start and stop delimit an observation interval, and event indicates whether the event occurs at the interval end. The displayed records show covariates updated across intervals for the same patient; stop must exceed start.

Slide 36: Using RHF with ordinary baseline survival data

RHF also accepts time-static problems after convert.counting transforms one-row-per-subject survival data into the required counting-process format. The peakVO2 example is converted, the four-column Surv formula is defined, and rhf is called exactly as it would be for genuinely time-varying data.

Slide 37: Checking a time-static RHF fit

The printed object reports 2,231 records from 2,231 unique subjects, 726 events, 39 features, and no time-varying features. The forest has 500 trees, an average of 15 leaves per tree, an average node size of 94, and hazard.loglik splitting with 10 random split points. The displayed OOB risk is 0.163. One record per subject confirms the time-static setting, even though the data use the counting-process interface.

Slides 38-39: Time-localized VarPro importance

importance.rhf calculates VarPro importance within a moving time window over the follow-up grid. Dot-matrix and line displays show how a feature's priority changes over time rather than imposing one global survival ranking. Slide 39 enlarges the dot-matrix and line displays so changes in variable priority across follow-up time are easier to read.

Closing (Slides 40-41)

Slide 40: Workshop outline

The closing roadmap returns to the four-module structure and shows how the advanced examples complete the course: class imbalance, train/test imputation and OOD scoring, SGT, and RHF all build on the core grow, OOB, prediction, and variable-selection workflows.

Slide 41: References

The references provide starting points for the RFQ imbalance method, random-forest missing-data algorithms, missForest, Super Greedy Trees, and the randomForestSGT and randomForestRHF vignettes. They support further study of the classification, imputation, geometric-splitting, and hazard-modeling examples in this module.